

Remote ATM Cash Withdrawal Automation Technical Report

1. Executive Summary

This report covers the technical architecture and implementation protocols of the remote cash withdrawal automation system, developed to ensure full integration of individuals with limited mobility (disabled, elderly, or bedridden) into the financial ecosystem. The project digitizes traditional ATM transactions by synchronizing a mobile banking session initiated by the user in home comfort with the physical presence of a field operator. The system's success is based on a hybrid architecture that bypasses detection mechanisms of high-security banking applications through advanced camera injection, GPS manipulation, and virtual Android environments.

2. Project Definition and Target Audience

The primary goal of the project is to enable disadvantaged user groups with physical access barriers to access cash through their own bank accounts without granting power of attorney to a third party or taking physical risks. The system offers an operational workflow where the "proxy operator" and the "actual user" remain linked; the user completes biometric and digital verification processes on their own device, while the physical cash dispense occurs in the field.

3. Technical Architecture and Infrastructure Components

As a system architect, the most critical decision for the project is the choice of hardware architecture. Traditional x86_64 servers remain insufficient for this type of operation due to "Architecture Conflict."

Native ARM64 Necessity:

Tier-1 banking applications (e.g., YKB) use high-security JNI (Java Native Interface) libraries and complex integrity checks. Translation layers used in x86_64-based systems (libhoudini or binfmt_misc) cannot fully simulate the ARM instruction sets expected by these libraries, causing the system to destabilize and trigger a **SIGSEGV (Segmentation Fault)** error. Therefore, it is an architectural necessity to host the solution on native ARM64 infrastructure such as Oracle Ampere A1 or AWS Graviton.

Software Layer and Virtualization:

Redroid (Docker-based Android), which offers a distributed structure, is run on Ubuntu 22.04 LTS. Appium, providing full control over the virtual device for workflow automation, has been preferred.

Resource and Stability:

A minimum of 16GB RAM configuration is required for enterprise-level uptime and simultaneous transaction capacity. This requirement is the "stability & concurrency" threshold for the operating system, Appium server, and resource-heavy banking applications to run simultaneously without stability loss.

4. Detailed Workflow and Synchronization Protocol

The transaction flow is based on a precise "handshake" logic between the server and the field:

1. **Remote Session:** The user connects to the Redroid instance on the central server via a secure tunnel.
2. **Initiating Transaction:** A "Withdraw Cash via QR" request is created through the banking app (e.g., YKB).
3. **Data Pooling:** Transaction ID and user data are kept in the virtual pool for matching.
4. **Field Capture:** The operator captures the physical QR code on the ATM screen in high resolution.
5. **Camera Injection (Handshake):** The image stream from the operator is processed via FFmpeg on the server side and directed to the `/dev/videoX` virtual device created via **V4L2Loopback**. Redroid perceives this virtual device as a physical hardware camera and injects the QR code into the application.
6. **GPS Synchronization:** For the application to pass location verification, GPS coordinates on the server side are synchronized with the ATM's physical location at the microsecond level.
7. **Finalization:** The bank server accepts both the QR data and the manipulated location data as "valid," triggering the ATM cash dispense command.

5. Security and Evasion (Bypass) Methods

Modern banking applications perform advanced "Environment Integrity" checks. To overcome these barriers, the following technical synergy is applied:

- **Zygisk:** Magisk's ability to inject into application processes forms the basic manipulation layer.
- **Shamiko:** Critical for hiding Zygisk traces. By running in "non-whitelist" mode, it ensures the root environment and Zygisk interface are completely hidden from the isolated processes of banking apps.
- **Hide My Applist:** Provides a "veiling" layer that prevents the application from seeing other automation (Appium) and analysis tools on the system.

The correct configuration of these tools creates a "stealth" environment where the application perceives itself to be on a standard user device.

6. Technical Requirements Table

Feature / Component	Technical Requirement / Tool	Rationale
Infrastructure	Oracle/AWS ARM - Ubuntu 22.04	Prevention of SIGSEGV errors with Native ARM64 support.

Feature / Component	Technical Requirement / Tool	Rationale
Android Environment	Redroid (Docker-based)	Container-based isolation and high-performance virtualization.
Automation	Appium (Python/Node.js)	UI navigation and programmatic control of transaction flow.
Camera Simulation	V4L2Loopback + FFmpeg	Mapping the virtual video stream to the physical hardware layer.
GPS Simulation	Kernel-level / LSPosed	Bypassing bank location verifications.
Security / Bypass	Zygisk + Shamiko + HMA	Neutralization of root and emulator detection mechanisms.

7. Technical Challenges and Lessons Learned

Critical Architectural Note: The biggest obstacle encountered during development is the security library incompatibility in x86 architecture. The obfuscated JNI structures of banking applications are designed to crash emulation layers directly. This proves the project is not just a software solution but requires hardware-level (Native ARM64) alignment.

Technical Note: Optimization of FFmpeg buffer settings to prevent latency in camera injection is vital for the QR code to be read by the application before timeout.

8. Conclusion and Implementation Scope

The system outlined in this report requires not only software development but also deep system security and kernel-level manipulation expertise. Operational success depends on the developer having experience in Linux Kernel modules, advanced bypass strategies, and automation pipelines.